

# MI SED API

---

**Version 3.0**

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

## REVISION HISTORY

| Revision No. | Description                                                     | Date       |
|--------------|-----------------------------------------------------------------|------------|
| 3.0          | <ul style="list-style-type: none"><li>Initial release</li></ul> | 06/16/2021 |

## TABLE OF CONTENTS

|                                       |           |
|---------------------------------------|-----------|
| <b>REVISION HISTORY</b>               | <b>i</b>  |
| <b>TABLE OF CONTENTS</b>              | <b>ii</b> |
| <b>1. API REFERENCE</b>               | <b>1</b>  |
| 1.1. Overview                         | 1         |
| 1.1.1 Image Detection Algorithms Type | 1         |
| 1.1.2 Role of SED                     | 2         |
| 1.1.3 Data Sources for SED            | 2         |
| 1.1.4 SED Detection Flow              | 2         |
| 1.1.5 Platform Limitations            | 3         |
| 1.2. Keyword Description              | 3         |
| 1.2.1 SED                             | 3         |
| 1.2.2 VDF                             | 3         |
| 1.2.3 IVE                             | 3         |
| 1.2.3 IPU                             | 3         |
| 1.2.4 ROI                             | 3         |
| 1.2.5 Motion                          | 4         |
| 1.2.6 QP                              | 4         |
| 1.2.7 MD                              | 5         |
| 1.2.8 OD                              | 5         |
| 1.2.9 VG                              | 5         |
| 1.2.10 SAD                            | 5         |
| 1.2.11 CBR                            | 5         |
| 1.2.12 VBR                            | 5         |
| 1.2.13 CVBR                           | 6         |
| 1.2.14 ABR                            | 6         |
| 1.2.15 AVBR                           | 6         |
| 1.2.16 IOU                            | 6         |
| <b>2. API List</b>                    | <b>8</b>  |
| 2.1. MI_SED_CreateChn                 | 9         |
| 2.2. MI_SED_DestroyChn                | 9         |
| 2.3. MI_SED_StartDetector             | 10        |
| 2.4. MI_SED_StopDetector              | 11        |
| 2.5. MI_SED_AttachToVencChn           | 11        |
| 2.6. MI_SED_DetachFromVencChn         | 12        |
| 2.7. MI_SED_GetRect                   | 13        |
| 2.8. MI_SED_SetDbgLevel               | 13        |
| <b>3. SED DATA TYPE</b>               | <b>15</b> |
| 3.1. SED_MAX_CHN_NUM                  | 16        |
| 3.2. SED_MAX_ROI_NUM_PER_CHN          | 16        |
| 3.3. SED_MAX_TARGET_CHN_NUM_PER_CHN   | 16        |
| 3.4. SED_MAX_CUS_DEF_ALGOPARAM_NUM    | 17        |
| 3.5. MI_SED_CHN                       | 17        |
| 3.6. MI_SED_TARGET_CHN                | 17        |

|           |                                    |           |
|-----------|------------------------------------|-----------|
| 3.7.      | MI_SED_AlgoType_e.....             | 18        |
| 3.8.      | MI_SED_InputAttr_t.....            | 18        |
| 3.9.      | MI_SED_IveObjDetect_Algo_t.....    | 19        |
| 3.10.     | MI_SED_CNNObjDetect_Algo_t.....    | 19        |
| 3.11.     | MI_SED_MotionObjDetect_Algo_t..... | 20        |
| 3.12.     | MI_SED_Rect_t.....                 | 20        |
| 3.13.     | MI_SED_RectInfo_t.....             | 21        |
| 3.14.     | MI_SED_AlgoAttr_t.....             | 22        |
| 3.15.     | MI_SED_TargetAttr_t.....           | 22        |
| 3.16.     | MI_SED_DetectorAttr_t.....         | 23        |
| <b>4.</b> | <b>ERROR CODE.....</b>             | <b>24</b> |
| 5.        | Demo.....                          | 25        |

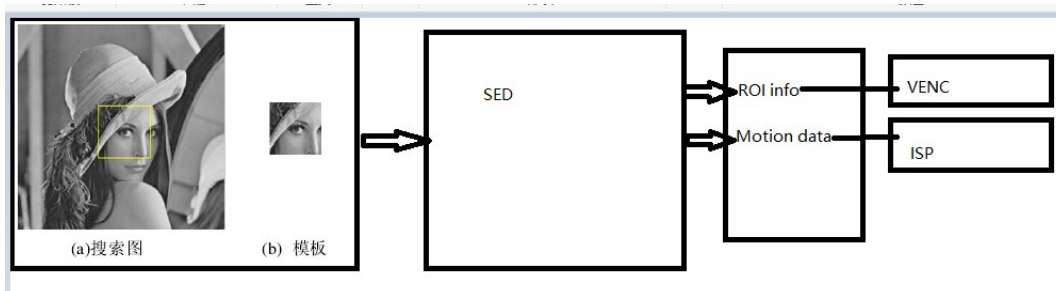
## 1. API REFERENCE

### 1.1. Overview

SED (smart coding)

The intelligent coding module mainly provides the functions of creating and destroying the intelligent coding channel, opening and stopping the detection source image, calculating the result and associating it with the specified coding channel.

The core function of intelligent coding is first to do image recognition (recognition of images of objects in motion, or recognition of objects in the image), and then to set the coding parameters to the venc module (to increase the QP value and improve clarity (sharpness) when movement of objects is detected, or to increase the QP value and reduce transmission bandwidth when the image detected is in stationary state). The detection algorithm will compare the difference between two frames to detect whether there is any movement of objects, extract the ROI and motion information of the image, and set the QP value of the specified ROI area for the encoder to do the encoding. There are many types of detection algorithms. Those utilized in our platform will be introduced below.



Through the intelligent identification of ROI by the SED, as well as the dynamic setting of QP value against the ROI designated by the image, we can effectively reduce bandwidth while ensuring the quality of the image.



### 1.1.1 Image Detection Algorithms Type

1: E\_MI\_IVEOBJDETECT\_ALGOPARAM: detects moving objects based on VDF/MD mode, with support for ROI image tracking.

2: E\_MI\_CNNOBJDETECT\_ALGOPARAM: objects which this algorithm can detect include bicycle, bus, car, motorbike, human beings. ROI image tracking is supported.

3: E\_MI\_MOTIONDETECT\_ALGOPARAM: motion detection based on IVE, supported only for AVBR motion detection.

The above three detection algorithms can be supported simultaneously by creating different detection channel instances.

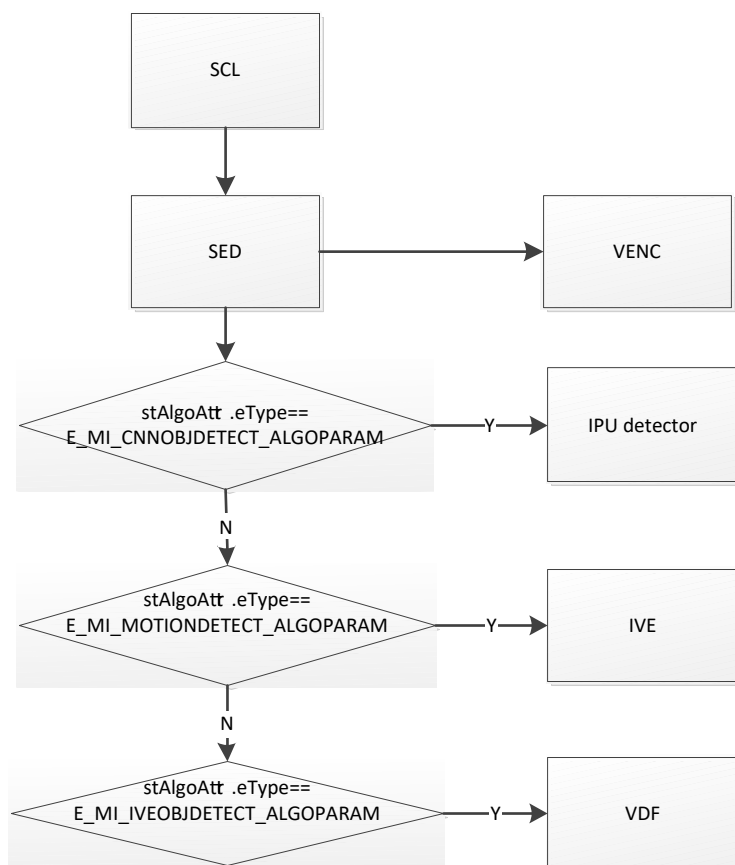
### 1.1.2 Role of SED

The SED is responsible for identifying ROI and motion information using Sigmastar or third-party intelligent algorithms. ROI information is used for VENC setting, and motion information is used for automatic adjustment of ISP sharpness, etc.

### 1.1.3 Data Sources for SED

SED obtains yuv data through SCL for intelligent detection (the resolution of yuv input is 352\*288).

### 1.1.4 SED Detection Flow



### 1.1.5 Platform Limitations

Human non-vehicle detection is based on the detection of AI training model through IPU. The accuracy of specific detection depends on the accuracy of AI model training. Currently, only the Pudding/Tiramisu platforms support human non-vehicle detection, while other low-order platforms do not.

## 1.2. Keyword Description

### 1.2.1 SED

The main functions of Smart Encoder (SED) are the creation and destruction of smart encoding channels, opening and stopping detection of source images, calculating results and linking to designated encoding channels, etc.

### 1.2.2 VDF

VDF (Video Detection Framework) realizes initialization of MD, OD and VG video channels, channel management, management of video detection results, channel destruction and other functions

### 1.2.3 IVE

IVE (Intelligent Video Engine) module is responsible for the processing and calculation of image data. The SAD value of two images can be obtained through MI\_IVE\_Sad, and the histogram statistical task data can be executed through MI\_IVE\_Hist to achieve the dynamic and static detection of images.

### 1.2.4 IPU

IPU (AI Process Unit) is an AI model processing and computing unit. The MI IPU module implements the rapid reasoning function of the network model, independently configuring the network model for each channel, and managing the acquisition and release of input and output data.

### 1.2.5 ROI

ROI (Region Of Interest) means coding against region of interest. With the ROI function enabled, important or mobile area, i.e. the ROI region, will be in high quality lossless coding. On the other hand, for those areas which remain motionless or which are not selected as the ROI areas, the bit rate and image quality will be reduced for standard definition video compression, or even omission of this part of video transmission, so as to ultimately save the network bandwidth and video storage space. Users can configure their ROI area to limit the area of image Qp, so as to differentiate the Qp of that area from the Qp of the other area in the image. The system supports both H.264 and H.265 encoding ROI settings and provides 16 ROI regions for simultaneous use by users.



The 16 ROI regions can overlap with one another, and the priority of the overlapped regions is increased in sequence according to the index number of 0-15, that is to say, the QP of the overlapped regions is finally determined to be processed only according to the highest priority regions. The ROI area can be configured with absolute QP and relative QP.

Absolute QP: The QP in ROI area is the QP value set by the user

Relative QP: The QP in ROI area is the sum of the QP generated by rate control and the QP offset value set by the user. In the following example, the encoded image adopts the FixQp mode, with the image QP set to 30, that is, the QP value of all macroblocks of the image is 30. ROI region 0 is set to absolute QP mode, with QP value = 20, index = 0; ROI region 1 is set to relative QP mode, with QP value = -15, index = 1. Because the index of ROI region 0 is smaller than the index of ROI region 1, the QP of ROI region 1 with higher priority is set in the overlapped image region. The QP value of region 1 is  $30 - 15 = 15$ , whereas the the QP value of region 0 is 20.



### 1.2.6 Motion

SAD information that contains the image.

### 1.2.7 QP

QP (Quantization Parameter) value corresponds to the sequence number of quantized step length. The smaller the value, the smaller the quantized step length, the higher the quantization accuracy, the better the picture quality, and the larger the size encoded.

### 1.2.8 MD

Motion detection is used to detect the movement of objects in films, and is applied to security monitoring in actual application.

### 1.2.9 OD

The occlusion detection is used to detect whether the received movie is blocked and to output the blocked detection result, if any.

### 1.2.10 VG

The virtual line segment is used to detect whether an object has crossed the set alarm line.  
A zone intrusion is used to detect if an object is passing through a set alarm area.

### 1.2.11 SAD

SAD means the Sum of Absolute Differences. This algorithm is often used for image block matching – the sum of the absolute value of the difference between the corresponding values of each pixel is used to evaluate the similarity of two image blocks. It can be seen that this algorithm is fast, but not accurate, and is usually used for preliminary screening of multistage processing.

The value range of SAD is [0, 255]. All SAD values are divided into 16 ranges:

[0, 10), [10, 15), [15, 20), [20, 25), [25, 30), [30, 40), [40, 50), [50, 60), [60, 70), [70, 80), [80, 90), [90, 100), [100, 120), [120, 140), [140, 160), [160, 255].

In this algorithm, the percentage of all MB (8\*8) in a frame that falls into the above range is calculated. The larger the value, the more static the picture.

### 1.2.12 CBR

CBR refers to Constant Bit Rate in case of Motion. Since the code rate is constant, the size of the code word can only be reduced by increasing QP, which will however cause the image quality to worsen. When the scene is still, the image quality becomes better; therefore, the image quality is generally unstable. This algorithm prioritizes bitrate (bandwidth) over other factors.

### 1.2.13 VBR

VBR (Variable Bit Rate) means the bit rate is dynamic, varying with the complexity of the image, so its coding efficiency is relatively high. When motion occurs, mosaic is rare. The bitrate control algorithm determines the bit rate to be used according to the image content. If the image content is simple, less bitrate will be allocated (it seems that the codeword is more suitable), and if the image content is complex, more codeword will be allocated, which not only guarantees the quality but also takes into account the bandwidth limit. This algorithm gives priority to image quality over other factors.

### 1.2.14 CVBR

CVBR (Constrained Variable Bit Rate) is an improvement on VBR. The Maximum bitRate or Average bitRate corresponding to this algorithm is constant. This method maintains the advantages of the above two methods: when the image content is still, the bandwidth is saved; when the image content is in motion, the bandwidth saved in the early stage is used to improve the image quality as much as possible, so as to achieve the purpose of retaining both bandwidth and image quality. This method usually allows the user to input the maximum and minimum bit rate. When still, the bit rate will be kept at the minimum bit rate, and when in motion, the bit rate will be greater than the minimum bit rate, but not more than the maximum bit rate.

### 1.2.15 ABR

The ABR (Average Bit Rate) refers to the average bit rate to reach the set code rate in a certain time range. The local peak code rate can exceed the set code rate, but the average code rate is constant.

### 1.2.16 AVBR

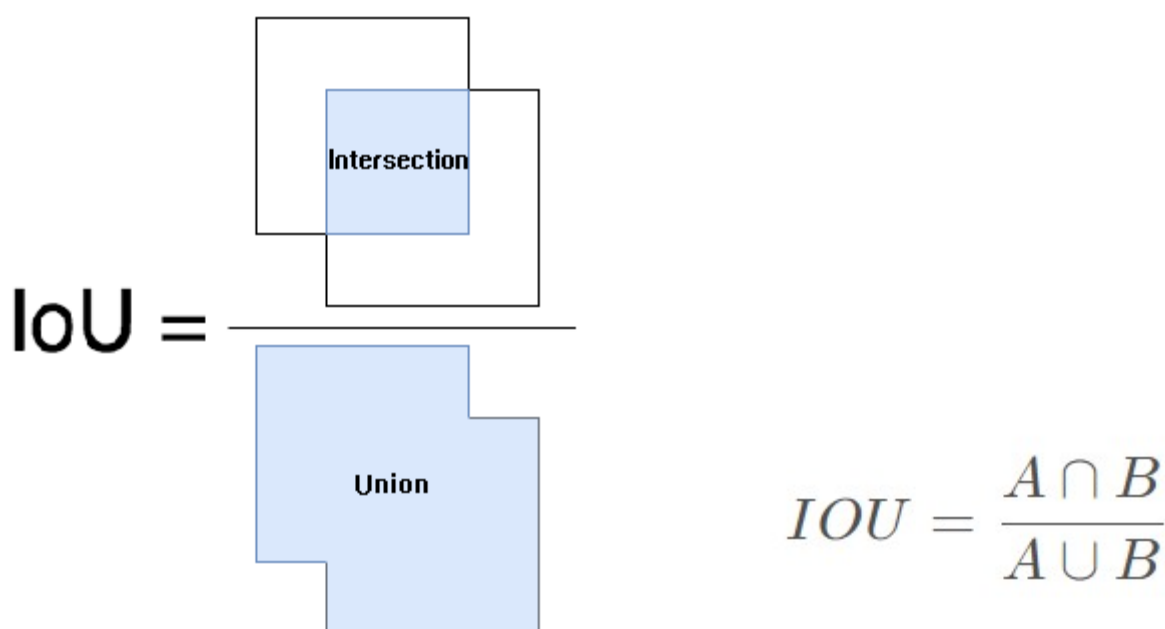
AVBR (Adaptive Variable Bit Rate) variable bit rate, which allows the code rate to fluctuate during the code rate statistical time, so as to ensure the stable quality of the coded image. The code rate control internally detects the motion and static state of the current scene, encodes with a higher code rate when moving, and actively reduces the target code rate when it is stationary.

Taking H.264 encoding as an example, the VENC module provides users with settings for MaxBitrate, ChangePos and MinStillPercent. MaxBitrate represents the maximum bit rate in a sports scene, and MaxBitrate\*ChangePos\*MinStillPercent represents the minimum bit rate in a static situation.

The target bit rate will be adjusted between the maximum bit rate and the minimum bit rate according to the degree of movement. MaxQp and MinQp are used to control the quality range of the image. The bit rate control takes QP clamping as the highest priority. If it exceeds MinQp, the bit rate control will be invalid in the MaxQp range.

### 1.2.17 IOU

The IOU (Intersection over Union) concept, used in target detection, means the overlapping rate of the generated candidate bound and the original ground truth bound, that is, the ratio of their intersection and union. (For more details, please go to: <http://172.19.30.188:8090/pages/viewpage.action?Pageid=1216493>)



The best case is complete overlap, i.e. the ratio is 1.

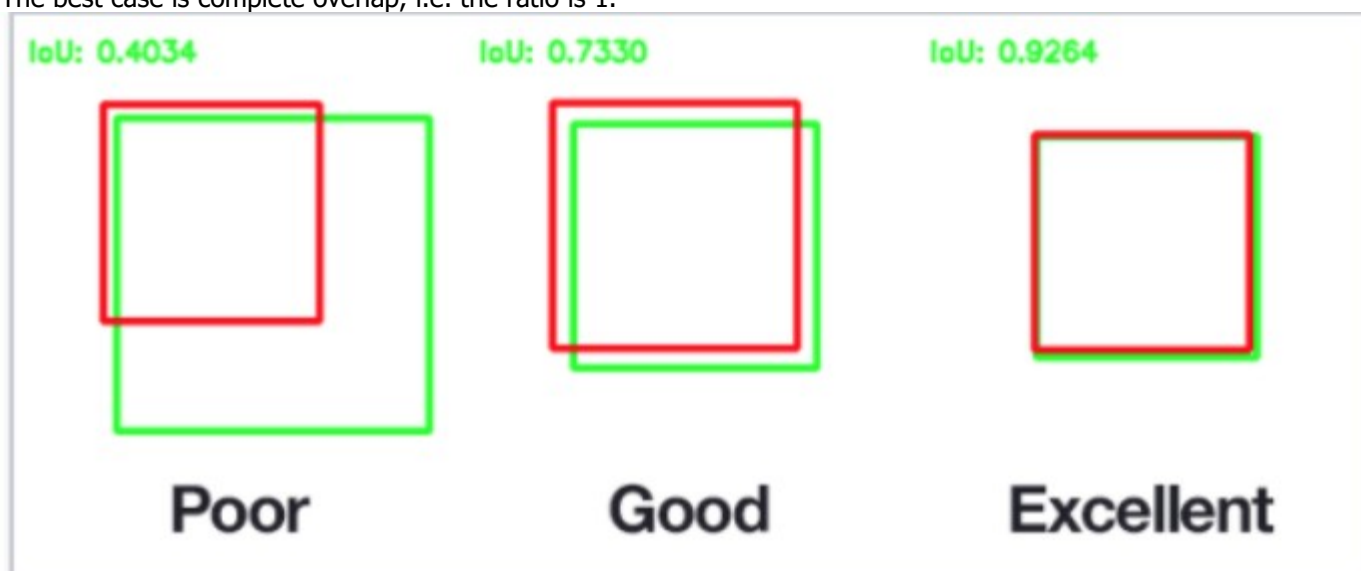


Diagram explaining IoU (from Wikipedia)

## 2. API LIST

---

This module provides the following APIs:

| API Name                                 | Function                                       |
|------------------------------------------|------------------------------------------------|
| <a href="#">MI_SED_CreateChn</a>         | Create an SED channel                          |
| <a href="#">MI_SED_DestroyChn</a>        | Destroy an SED channel                         |
| <a href="#">MI_SED_StartDetector</a>     | Start detection of designated channel          |
| <a href="#">MI_SED_StopDetector</a>      | Stop detection of designated channel           |
| <a href="#">MI_SED_AttachToVencChn</a>   | Attach module to designated venc channel       |
| <a href="#">MI_SED_DetachFromVencChn</a> | Detach module from designated venc channel     |
| <a href="#">MI_SED_GetRect</a>           | Get the rect info data detected by SED         |
| <a href="#">MI_SED_SetDbgLevel</a>       | Set SED module debug level, control log output |

## 2.1. MI\_SED\_CreateChn

### ➤ Function

Create an SED channel

### ➤ Syntax

```
MI_S32 MI_SED_CreateChn(MI\_SED\_CHN SedChn, MI\_SED\_DetectorAttr\_t* pstAttr);
```

### ➤ Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |
| pstAttr        | SED detection attribute pointer                                     | Input        |

### ➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

### ➤ Dependency

- Header: mi\_common.h, mi\_sed.h
- Library: libmi\_sed.a

### ➤ Note

N/A.

### ➤ Example

N/A.

### ➤ Related Function

N/A.

## 2.2. MI\_SED\_DestroyChn

### ➤ Function

Destroy an SED channel.

### ➤ Syntax

```
MI_S32 MI_SED_DestroyChn(MI\_SED\_CHN SedChn);
```

### ➤ Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see [error code for](#) details
- Dependency
  - Header: mi\_common.h, mi\_sed.h
  - Library: libmi\_sed.a
- Note
 

N/A.
- Example
 

N/A.
- Related Function
 

N/A.

## 2.3. MI\_SED\_StartDetector

- Function
 

Start detection of a designated channel.
- Syntax
 

```
MI_S32 MI_SED_StartDetector(MI\_SED\_CHN SedChn);
```
- Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see [error code for](#) details
- Dependency
  - Header: mi\_common.h, mi\_sed.h
  - Library: libmi\_sed.a
- Note
 

N/A.
- Example
 

N/A.
- Related Function
 

N/A.

## 2.4. MI\_SED\_StopDetector

➤ Function

Stop detection of a designated channel.

➤ Syntax

MI\_S32 MI\_SED\_StopDetector([MI\\_SED\\_CHN](#) SedChn);

➤ Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi\_common.h, mi\_sed.h
- Library: libmi\_sed.a

➤ Note

N/A.

➤ Example

N/A.

➤ Related Function

N/A.

## 2.5. MI\_SED\_AttachToVencChn

➤ Description

Attach module to designated venc channel.

➤ Syntax

MI\_S32 MI\_SED\_AttachToChn([MI\\_SED\\_CHN](#) SedChn, [MI\\_SED\\_TARGET\\_CHN](#) TargetChn);

➤ Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |
| TargetChn      | Target encoder channel<br>Range: [0, VENC_MAX_CHN_NUM)              | Input        |

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see [error code for](#) details
- Dependency
  - Header: mi\_common.h, mi\_sed.h
  - Library: libmi\_sed.a
- Note
 

N/A.
- Example
 

N/A.
- Related Function
 

N/A.

## 2.6. [MI\\_SED\\_DetachFromVencChn](#)

- Description
 

Detach module from designated venc channel.
- Syntax
 

```
MI_S32 MI_SED_DetachFromChn(MI\_SED\_CHN SedChn, MI\_SED\_TARGET\_CHN TargetChn);
```

- Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |
| TargetChn      | Target encoder channel.<br>Range: [0,VENC_MAX_CHN_NUM)              | Input        |

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see [error code for](#) details
- Dependency
  - DependencyHeader: mi\_common.h, mi\_sed.h
  - Library: libmi\_sed.a
- Note
 

N/A.
- Example
 

N/A.

- Related Function  
N/A.

## 2.7. MI\_SED\_GetRect

- Description  
Get the rect info data detected by sed
- Syntax  
MI\_S32 MI\_SED\_GetRect(MI\_SED\_CHN SedChn, MI\_SED\_RectInfo\_t \*pstRectInfo)

- Parameter

| Parameter Name | Description                                                         | Input/Output |
|----------------|---------------------------------------------------------------------|--------------|
| SedChn         | SED channel number.<br>Range: [0, <a href="#">SED_MAX_CHN_NUM</a> ) | Input        |
| pstRectInfo    | Rect info                                                           | Output       |

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see [error code for](#) details
- Dependency
  - DependencyHeader: mi\_common.h, mi\_sed.h
  - Library: libmi\_sed.a
- Note  
N/A.
- Example  
N/A.
- Related Function  
N/A.

## 2.8. MI\_SED\_SetDbgLevel

- Description  
Set SED module debug level, control log output
- Syntax  
MI\_S32 MI\_SED\_SetDbgLevel(MI\_DBG\_LEVEL\_e eLevel)

➤ Parameter

| Parameter Name | Description                                    | Input/Output |
|----------------|------------------------------------------------|--------------|
| eLevel         | MI debug level enum.<br>Range: [0, MI_DBG_ALL] | Input        |

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- DependencyHeader: mi\_common.h, mi\_sed.h
- Library: libmi\_sed.a

➤ Note

N/A.

➤ Example

N/A.

➤ Related Function

N/A.

## 1. SED DATA TYPE

---

The relevant data types and data structures are defined as follows:

| DATA TYPE                                      | Definition                                                                 |
|------------------------------------------------|----------------------------------------------------------------------------|
| <a href="#">SED_MAX_CHN_NUM</a>                | Define maximum SED channel number                                          |
| <a href="#">SED_MAX_ROI_NUM_PER_CHN</a>        | Define maximum ROI number per channel                                      |
| <a href="#">SED_MAX_TARGET_CHN_NUM_PER_CHN</a> | Define maximum target VENC channel number per SED channel                  |
| <a href="#">SED_MAX_CUS_DEF_ALGOPARAM_NUM</a>  | Define maximum number of supported customer-defined algorithm              |
| <a href="#">MI_SED_CHN</a>                     | Define SED channel number                                                  |
| <a href="#">MI_SED_TARGET_CHN</a>              | Define VENC encoder channel number corresponding to the SED channel number |
| <a href="#">MI_SED_AlgoType_e</a>              | Define SED algorithm type                                                  |
| <a href="#">MI_SED_InputAttr_t</a>             | Define SED input attribute                                                 |
| <a href="#">MI_SED_IveObjDetect_Algo_t</a>     | Do IVE Detector Param                                                      |
| <a href="#">MI_SED_CNNObjDetect_Algo_t</a>     | Do CNN Object Detect Param                                                 |
| <a href="#">MI_SED_MotionObjDetect_Algo_t</a>  | Define SED input attribute                                                 |
| <a href="#">MI_SED_Rect_t</a>                  | Get SED detector rect data                                                 |
| <a href="#">MI_SED_RectInfo_t</a>              | All rectangles info detected by SED                                        |
| <a href="#">MI_SED_AlgoAttr_t</a>              | Define SED algorithm properties                                            |
| <a href="#">MI_SED_TargetAttr_t</a>            | Define the final effective coding target attribute                         |
| <a href="#">MI_SED_DetectorAttr_t</a>          | Define SED detection properties                                            |

## 1.1. SED\_MAX\_CHN\_NUM

- Description  
Define maximum SED channel number.
- Definition  
`#define SED_MAX_CHN_NUM (64)`
- Note  
N/A.
- Related Data Type and Interface  
N/A.

## 1.2. SED\_MAX\_ROI\_NUM\_PER\_CHN

- Description  
Define maximum ROI number per channel.
- Definition  
`#define SED_MAX_ROI_NUM_PER_CHN (16)`
- Note  
N/A.
- Related Data Type and Interface  
N/A.

## 1.3. SED\_MAX\_TARGET\_CHN\_NUM\_PER\_CHN

- Description  
Define maximum target VENC channel number per SED channel.
- Definition  
`#define SED_MAX_TARGET_CHN_NUM_PER_CHN (8)`
- Note  
N/A.
- Related Data Type and Interface  
N/A.

## 1.4. SED\_MAX\_CUS\_DEF\_ALGOPARAM\_NUM

- Description  
Define maximum number of supported customer-defined algorithm.
- Definition  
`#define SED_MAX_CUS_DEF_ALGOPARAM_NUM (10)`
- Note  
N/A.
- Related Data Type and Interface  
N/A.

## 1.5. MI\_SED\_CHN

- Description  
Define SED channel number.
- Definition  
`typedef MI_S32 MI_SED_CHN;`
- Note  
N/A.
- Related Data Type and Interface  
N/A.

## 1.6. MI\_SED\_TARGET\_CHN

- Description  
Define VENC encoder channel number corresponding to the SED channel number.
- Definition  
`typedef MI_S32 MI_SED_TARGET_CHN;`
- Note  
N/A.
- Related Data Type and Interface  
N/A.

## 1.7. MI\_SED\_AlgoType\_e

➤ Description

Define SED algorithm type.

➤ Definition

```
typedef enum
{
    E_MI_DEFAULT_ALGOPARAM = 0x0,
    E_MI_CUSDEF_ALGOPARAM = 0x1,
    E_MI_DEFAULT_ALGOPARAM_MAX
} MI_SED_AlgoType_e;
```

➤ Member

| Member Name            | Description                     |
|------------------------|---------------------------------|
| E_MI_DEFAULT_ALGOPARAM | Use default SDK algorithm       |
| E_MI_CUSDEF_ALGOPARAM  | Use customer-defined algorithm. |

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

## 1.8. MI\_SED\_InputAttr\_t

➤ Description

Define SED input attribute.

➤ Definition

```
typedef struct MI_SED_InputAttr_s
{
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_U32 u32FrameRateNum;
    MI_U32 u32FrameRateDen;
    MI_SYS_ChnPort_t stInputPort;
} MI_SED_InputAttr_t;
```

➤ Member

| Member Name     | Description                     |
|-----------------|---------------------------------|
| u32Width        | Input YUV width                 |
| u32Height       | Input YUV height                |
| u32FrameRateNum | Input YUV framerate numerator   |
| u32FrameRateDen | Input YUV framerate denominator |
| stInputPort     | Input YUV channel attribute     |

➤ Note

N/A.

➤ Related Data Type and Interface

MI\_SED\_CreateChn

MI\_SED\_DetectorAttr\_t

## 1.9. MI\_SED\_IveObjDetect\_Algo\_t

➤ Description

Do IVE Detector Param

➤ Definition

```
typedef struct MI_SED_IveObjDetect_Algo_s
{
    MI_U32      u32VdfChn;
    MI_U8       u8Sensitivity;
} MI_SED_IveObjDetect_Algo_t;
```

➤ Member

| Member Name   | Description           |
|---------------|-----------------------|
| u32VdfChn     | Vdf channel number    |
| u8Sensitivity | Algorithm sensitivity |

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

## 1.10. MI\_SED\_CNNObjDetect\_Algo\_t

➤ Description

Do CNN Object Detect Param

➤ Definition

```
typedef struct MI_SED_CNNObjDetect_Algo_s
{
    MI_U32 u32MaxVariableBufSize;
    MI_U32 u32IPUChnId;
    MI_U8 *u8FwImagePath;
    MI_U8 *u8ModelImagePath;
} MI_SED_CNNObjDetect_Algo_t;
```

➤ Member

| Member Name           | Description                              |
|-----------------------|------------------------------------------|
| u32MaxVariableBufSize | Maximum memory required by IPU algorithm |
| u32IPUChnId           | IPU Channel Number                       |
| u8FwImagePath         | IPU firmware file path                   |
| u8ModelImagePath      | AI Model file path                       |

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

## 1.11. MI\_SED\_MotionObjDetect\_Algo\_t

➤ Description

Define SED input attribute.

➤ Definition

```
typedef struct MI_SED_InputAttr_s
{
    MI_IVE_HANDLE iveHandle;
} MI_SED_InputAttr_t;
```

➤ Member

| Member Name | Description                         |
|-------------|-------------------------------------|
| iveHandle   | IVE file handle, default value is 0 |

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

## 1.12. MI\_SED\_Rect\_t

➤ Description

Get SED detector rect data

## ➤ Definition

```
typedef struct MI_SED_Rect_s
{
    MI_U32 u32Left;
    MI_U32 u32Top;
    MI_U32 u32Width;
    MI_U32 u32Height;
} MI_SED_Rect_t;
```

## ➤ Member

| Member Name | Description                                                  |
|-------------|--------------------------------------------------------------|
| u32Left     | Column coordinates of the upper left corner of the rectangle |
| u32Top      | Row coordinates of the upper left corner of the rectangle    |
| u32Width    | Width of rectangle                                           |
| u32Height   | Height of rectangle                                          |

## ➤ Note

N/A.

## ➤ Related Data Type and Interface

N/A.

### 1.13. MI\_SED\_RectInfo\_t

## ➤ Description

All rectangle info detected by SED

## ➤ Definition

```
typedef struct MI_SED_RectInfo_s
{
    MI_U32 u32RectCount;
    MI_SED_Rect_t stRect[SED_MAX_ROI_NUM_PER_CHN];
} MI_SED_RectInfo_t;
```

## ➤ Member

| Member Name  | Description                    |
|--------------|--------------------------------|
| u32RectCount | Number of rectangles           |
| stRect       | Information of rectangular box |

## ➤ Note

N/A.

## ➤ Related Data Type and Interface

N/A.

## 1.14. MI\_SED\_AlgoAttr\_t

### ➤ Description

Define SED algorithm properties

### ➤ Definition

```
typedef struct MI_SED_CusAlgoAttr_s
{
    MI_SED_AlgoType_e eType;
    Union
    {
        MI_SED_IveObjDetect_Algo_t stIveObjDetectAlgo;
        MI_SED_CNNObjDetect_Algo_t stCNNObjDetectAlgo;
        MI_SED_MotionObjDetect_Algo_t stMotionObjDetectAlgo;
    };
} MI_SED_AlgoAttr_t;
```

### ➤ Member

| Member Name           | Description                   |
|-----------------------|-------------------------------|
| eType                 | Type of algorithm used by SED |
| stIveObjDetectAlgo    | IVE algorithm parameters      |
| stCNNObjDetectAlgo    | CNN algorithm parameters      |
| stMotionObjDetectAlgo | Motion algorithm parameters   |

### ➤ Note

N/A.

### ➤ Related Data Type and Interface

N/A.

## 1.15. MI\_SED\_TargetAttr\_t

### ➤ Description

Define the final effective coding target attribute

### ➤ Definition

```
typedef struct MI_SED_TargetAttr_s
{
    MI_S32 s32RltQp;
} MI_SED_TargetAttr_t;
```

### ➤ Member

| Member Name | Description           |
|-------------|-----------------------|
| s32RltQp    | ROI relative QP value |

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

## 1.16. MI\_SED\_DetectorAttr\_t

➤ Description

Define SED detection properties

➤ Definition

```
typedef struct MI_SED_DetectorAttr_s
{
    MI_SED_InputAttr_t stInputAttr;
    MI_SED_AlgoAttr_t stAlgoAttr;
    MI_SED_TargetAttr_t stTargetAttr;
} MI_SED_DetectorAttr_t;
```

➤ Member

| Member Name  | Description                                  |
|--------------|----------------------------------------------|
| stInputAttr  | Properties of the SED input section          |
| stAlgoAttr   | Attributes related to SED algorithm          |
| stTargetAttr | Target control attribute of SED final effect |

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

## 2. ERROR CODE

---

The SED API error codes are shown in the table below:

Table 4-1: SED API Error Code

| Error Code | Macro Definition           | Description                       |
|------------|----------------------------|-----------------------------------|
| 0x00000000 | MI_SUCCESS                 | Success                           |
| 0xA01E2002 | MI_ERR_SED_INVALID_CHNID   | Invalid channel ID                |
| 0xA01E2003 | MI_ERR_SED_ILLEGAL_PARAM   | At lease one parameter is illegal |
| 0xA01E2004 | MI_ERR_SED_EXIST           | Channel already exists            |
| 0xA01E2005 | MI_ERR_SED_UNEXIST         | Channel does not exist            |
| 0xA01E2006 | MI_ERR_SED_NULL_PTR        | Using a NULL point                |
| 0xA01E200c | MI_ERR_SED_NOMEM           | Failure caused by malloc memory   |
| 0xA01E2013 | MI_ERR_SED_CHN_NOT_STARTED | Channel not started               |
| 0xA01E2014 | MI_ERR_SED_CHN_NOT_STOPPED | Channel not stopped               |
| 0xA01E2017 | MI_ERR_SED_NOT_ENABLE      | Channel not enabled               |

### **3. DEMO**

---

Build Demo code:

```
cd sdk/verify/mi_demo/alderaan$
```

```
make
```

```
get execute program file: sdk\verify\mi_demo\out\demo\app\prog_sed
```